

Stuart-Landau Oscillatory Graph Neural Network

Kaicheng Zhang

K.Zhang-60@sms.ed.ac.uk

School of Mathematics and Maxwell Institute

University of Edinburgh

Edinburgh, UK

Piero Deidda

piero.deidda@sns.it

Gran Sasso Science Institute

L'Aquila, Italy

Scuola Normale Superiore

Pisa, Italy

David N. Reynolds

d.reynolds@uw.edu.pl

Institute of Applied Mathematics and Mechanics

University of Warsaw

Warsaw, Poland

Francesco Tudisco

f.tudisco@ed.ac.uk

School of Mathematics and Maxwell Institute,

University of Edinburgh,

Edinburgh, UK

Miniml.AI

Edinburgh, UK

Abstract

Oscillatory Graph Neural Networks (OGNNs) are an emerging class of physics-inspired architectures designed to mitigate oversmoothing and vanishing gradient problems in deep GNNs. In this work, we introduce the Complex-Valued Stuart-Landau Graph Neural Network (SLGNN), a novel architecture grounded in Stuart-Landau oscillator dynamics. Stuart-Landau oscillators are canonical models of limit-cycle behavior near Hopf bifurcations, which are fundamental to synchronization theory and are widely used in e.g. neuroscience for mesoscopic brain modeling. Unlike harmonic oscillators and phase-only Kuramoto models, Stuart-Landau oscillators retain both amplitude and phase dynamics, enabling rich phenomena such as amplitude regulation and multistable synchronization. The proposed SLGNN generalizes existing phase-centric Kuramoto-based OGNNs by allowing node feature amplitudes to evolve dynamically according to Stuart-Landau dynamics, with explicit tunable hyperparameters (such as the Hopf-parameter and the coupling strength) providing additional control over the interplay between feature amplitudes and network structure. We conduct extensive experiments across node classification, graph classification, and graph regression tasks, demonstrating that SLGNN outperforms existing OGNNs and establishes a novel, expressive, and theoretically grounded framework for deep oscillatory architectures on graphs. The code and hyperparameters for SLGNN are available at <https://github.com/kevvezhang/StuartLandauGNN>

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Mathematics of computing** → *Ordinary differential equations*; *Graph algorithms*.

Keywords

Graph Neural Networks, Physics-inspired Neural Networks, Oscillatory Neural Networks, Neural ODEs

ACM Reference Format:

Kaicheng Zhang, David N. Reynolds, Piero Deidda, and Francesco Tudisco. 2026. Stuart-Landau Oscillatory Graph Neural Network. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792675>

1 Introduction

Graph neural networks (GNNs) are a class of neural networks designed to learn from graph-structured data by modeling the mutual influence between each node and its neighbors. These architectures have demonstrated effectiveness across diverse domains, ranging from social network analysis [9] to molecular property prediction [11] and recommendation systems [5, 31].

However, classical GNN architectures, such as graph convolutional networks (GCN) [16] and graph attention networks (GAT) [50], suffer from multiple pathologies stemming from their message-passing paradigm for aggregating features from neighboring nodes. These limitations include oversmoothing [27, 28, 39, 54], overcorrelation [37], oversquashing [12, 25], and vanishing gradients [40].

To address these challenges, numerous alternative architectures have been proposed, such as Graph Neural ODEs that inherit properties from the physical systems on which they are based [3, 6, 33, 49, 53]. Among them, a particularly promising class is the oscillatory graph neural networks (OGNN), based on coupled oscillator systems [26, 40]. In these models, each node feature is represented as an oscillator with coupling factors determined by the graph structure, drawing inspiration from models of electrical activity in biological neural networks. These architectures have been shown to mitigate several pathologies, including vanishing and exploding gradients as well as oversmoothing [26, 40].

In this work, we introduce the Stuart-Landau Graph Neural Network (SLGNN), a novel architecture based on complex-valued



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792675>

coupled Stuart-Landau oscillator dynamics. Stuart-Landau oscillators are canonical models of limit-cycle behavior near Hopf bifurcations, generalizing the well-known Kuramoto model of coupled oscillators. Critically, while the Kuramoto model evolves only phase (frequency) dynamics, the Stuart-Landau framework captures both amplitude and phase evolution. This fundamental difference gives rise to substantially richer dynamics: while Kuramoto models converge only to frequency-synchronized equilibrium states, Stuart-Landau systems exhibit a diverse repertoire of stable states—including frequency-synchronized and asynchronous configurations, amplitude-death and active states—whose emergence depends on the interplay between system hyperparameters. This increased expressiveness, rooted in the model’s ability to capture the nonlinear amplitude-stabilization mechanisms observed in biological neural oscillators, translates into enhanced learning capacity for SLGNN compared to existing OGNN models.

The contributions of this paper include the following:

- We provide a comprehensive overview of three oscillator types (Stuart-Landau, Kuramoto, and harmonic), detailing their dynamical properties and the implications on the learning capabilities.
- We introduce SLGNN, a new OGNN framework that models both amplitude and phase dynamics through Stuart-Landau coupled oscillators, enabling richer behaviors beyond the Kuramoto- or harmonic-based OGNNs.
- We evaluate the architecture using a bespoke IMEX time-stepping scheme that achieves 100× speedup over adaptive solvers while maintaining stability and performance.
- Through extensive experiments across node classification, graph classification, and graph regression tasks, we demonstrate that SLGNN achieves state-of-the-art performance by effectively leveraging deeper architectures.

2 Related Work

Continuous-depth GNN models based on graph ordinary differential equations (ODEs) have demonstrated significant success in circumventing known GNN limitations. Standard architectures like GATs can be interpreted as explicit discretizations of diffusion equations on graphs, and the corresponding continuous-time Graph Neural Diffusion (GRAND) model [3] has been shown to suffer substantially less from pathologies such as oversmoothing observed in discrete counterparts. Further improvements arise from augmenting the diffusion dynamics with reaction and advection terms, which can be rigorously proven to prevent convergence to node-wise constant steady states [4, 7, 8, 49]. Indeed, from the perspective of a dynamical system, the increasing similarity of the features in the long-time asymptotic is often a synonym of convergence of the trajectories of the different nodes toward a single steady state. This situation is directly linked to oversmoothing and always occurs when considering diffusion systems on a connected graph, in which case the unique steady state is the uniform one with magnitude depending on the initial condition. However, if the diffusion operator is learned in such a way as to almost disconnect the graph according to the task at hand, then convergence to the stable equilibrium can be significantly slowed. Additionally, adding reaction or advection terms to the model makes the steady state of

the system dependent on the initial one, thus avoiding phenomena like oversmoothing and overcorrelation.

A particularly promising class of continuous Neural ODEs consists of physics-inspired oscillatory models. Coupled Oscillatory Recurrent Neural Network (coRNN) [41] employs coupled damped second-order harmonic oscillators to mitigate the vanishing and exploding gradient problems in residual neural networks for tackling long-sequence tasks. Unified Invertible Coupled Oscillatory RNN (UnICORNN) [42] generalizes coRNN as a Hamiltonian system, improving memory efficiency while ensuring invertibility in time. Building on this, Linear Oscillatory State-Space models (LinOSS) [43] translates these dynamics from the standard RNN setting into a linear state-space model, enabling highly efficient computation via parallel scans rather than step-by-step recurrence. Lanthaler et al. [18] further demonstrates the universality of the harmonic oscillatory networks for approximating underlying functions. Apart from harmonic-oscillator-based approaches, Artificial Kuramoto Oscillatory Neurons (AKOrN) [21] propose a general-purpose oscillatory neuron as unit vectors evolving according to Kuramoto dynamics and with great adaptability with existing layer architectures such as convolutions and attention mechanisms.

For the application of oscillatory models to graph tasks, the Graph Coupled Oscillator Network (GraphCON) [40] uses damped harmonic oscillators similar to those of coRNN, while Kuramoto Graph Neural Networks (KuramotoGNN) [26] model each feature as a fixed-amplitude oscillator with nonlinear coupling between oscillators. Both approaches provably reduce or eliminate oversmoothing as well as exploding and vanishing gradient problems. Notably, in coupled oscillator systems, a steady state is always the one where both phases and amplitudes are synchronized and possibly zero if damping factors are included. Therefore, in order to avoid oversmoothing/overcorrelation problems, oscillatory models admitting steady states other than the fully synchronized one are desirable. In this setting, our proposed Stuart-Landau architecture is of particular interest. Indeed, by enabling dynamic amplitude evolution alongside phase dynamics, it enriches the space of possible equilibrium states and enhances model expressiveness.

3 Three types of neural oscillators

In this section, we present the Stuart-Landau oscillator dynamical system and provide an overview of two other prominent types of oscillators used for modeling physics-inspired graph neural networks: the Kuramoto oscillator and the (un)damped harmonic oscillator. We discuss their main properties and compare their relative benefits and deficits.

3.1 The Stuart-Landau Oscillator

A Stuart-Landau oscillator describes the behavior of a limit-cycle oscillator near a Hopf bifurcation. The equation is given by

$$\dot{z}(t) = (\alpha + i\omega - (\beta + i\gamma)|z(t)|^2)z(t), \quad (1)$$

$$\alpha, \omega, \beta, \gamma \in \mathbb{R}, \quad z(t) \in \mathbb{C}.$$

A Hopf bifurcation occurs when at a phase-transition a fixed point gives rise to a stable limit cycle. To see how each of the four parameters $(\alpha, \omega, \beta, \gamma)$ play a role in the dynamics and asymptotic outcomes let us break down the oscillator into its amplitude and

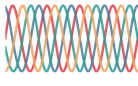
Oscillator Type	Magnitude Behaviour			Magnitude Convergence		Phase Behaviour	
	Amplitude Death	Orbit	Amplitude Growth	Algebraic	Exponential	Incoherence	Synchronized
							
Non-oscillatory	✓	✗	✗	✗	✗	✗	✗
Harmonic	✓	✓	✗	✗	✓	✓	✓
Kuramoto	✗	✓	✗	✗	✗	✓	✓
Stuart Landau	✓	✓	✓	✓	✓	✓	✓

Table 1: Dynamics that can be modeled by different oscillators and the non-oscillatory baseline. The magnitude behaviour and convergence figures show the dynamics of the oscillator on a complex plane. The black dot indicates the origin, and the dashed lines indicate some magnitude equilibrium position. The phase behaviour figures show the evolution of the phase over time.

phase values $z(t) = r(t)e^{i\phi(t)} \in \mathbb{C}$. The amplitude equation is given by

$$\dot{r}(t) = (\alpha - \beta r^2(t))r(t). \quad (2)$$

Only two parameters play a role in the amplitude behavior, which gives us 4 main cases, [20, 29, 36]:

- (1) ($\alpha > 0, \beta > 0$): There is a fixed point of (2) at $r_\infty = \sqrt{\alpha/\beta}$ which is stable and $r_\infty = 0$ which is unstable.
- (2) ($\alpha < 0, \beta \geq 0$): There is a stable fixed point at $r_\infty = 0$ for which all $r(t)$ converge to exponentially fast.
- (3) ($\alpha > 0, \beta \leq 0$): There is a fixed point at $r_\infty = 0$ which is unstable, and if $r_0 > 0$, then the amplitude experiences unbounded growth, $r(t) \rightarrow \infty$.
- (4) ($\alpha < 0, \beta < 0$): The fixed point $r_\infty = 0$ is still stable, but the basin of attraction is confined to $r_0^2 < \alpha/\beta$, while $r_\infty^2 = \alpha/\beta$ is an unstable fixed point where if $r_0^2 > \alpha/\beta$, then $r(t) \rightarrow \infty$.

The standard form for a Stuart-Landau oscillator assumes $\beta > 0$ to avoid situations in which the amplitude undergoes unbounded growth. This leads us to focus on cases (1) and (2). Case (1) can be identified with the Orbit and Amplitude Growth columns of Table 1, with magnitude convergence occurring at an exponential rate. Case (2) leads to Amplitude Death, again at an exponential rate. However, in the above four cases we left off the case of $\alpha = 0$ as, α , known as the Hopf-parameter has its critical value at zero. The critical case gives

$$\dot{r}(t) = -\beta r^3(t) \quad (3)$$

For $\beta > 0$, solving (3) gives

$$r(t) = (2\beta t + c)^{-1/2},$$

we don't see exponential convergence, but rather convergence to zero at an algebraic rate as in Table 1.

Now let us analyze the phase behavior,

$$\dot{\phi}(t) = \omega - \gamma r^2.$$

Again, the phase equation only depends on two parameters, however the dependence on the amplitude also plays a role. The parameter ω is known as the natural frequency of the oscillator and in the subcritical regime ($\alpha < 0, \beta > 0$) since $r(t) \rightarrow 0$ exponentially

fast the resulting phase dynamics lead to $\dot{\phi}(t) \rightarrow \omega$ also exponentially fast giving an almost constant phase speed as the oscillator converges to zero. The parameter γ is known as the phase shift parameter and when $\gamma = 0$ we see that $\dot{\phi} = \omega$ remains fixed, while for $\gamma > 0$ the oscillator is slowed down and respectively sped up for $\gamma < 0$. Now, if we are in the supercritical regime ($\alpha > 0, \beta > 0$) we have $\dot{\phi} \rightarrow \omega - \gamma \frac{\alpha}{\beta}$ so that γ provides for a phase shift reliant also on the amplitude parameters α and β .

Again, focusing on the critical case of $\alpha = 0, \beta > 0$ we see that $r^2(t) = (2\beta t + c)^{-1}$ and hence

$$\dot{\phi}(t) = \omega - \frac{\gamma}{2\beta t + c}, \quad (4)$$

which again implies that the phase settles to the natural frequency ω , as $t \rightarrow \infty$, but at an algebraic rate.

3.1.1 Coupled Stuart-Landau Oscillators. Coupling identical oscillators of the form (1) gives the following equation

$$\dot{z}_j = (\alpha + i\omega - (\beta + i\gamma)|z_j|^2)z_j + \sum_{l=1}^N A_{lj}(z_l - z_j), \quad (5)$$

for $j = 1, \dots, N$, where A represents the connectivity matrix. Separating into their respective amplitude and phase equations gives

$$\begin{aligned} \dot{r}_j &= (\alpha - \beta r_j^2)r_j + \sum_{l=1}^N A_{lj}(\cos(\phi_l - \phi_j)r_l - r_j), \\ \dot{\phi}_j &= \omega - \gamma r_j^2 + \sum_{l=1}^N A_{lj} \frac{r_l}{r_j} \sin(\phi_l - \phi_j). \end{aligned} \quad (6)$$

When the system is decoupled and each of the N oscillators converge to their limit-cycle (supercritical case) or to the zero fixed point (subcritical case). In the supercritical case, the initial data of each oscillator will determine where on the limit-cycle each oscillator can be found. Once the coupling is activated, again in the supercritical case, if the connectivity matrix is connected, one can observe that

$$\begin{aligned} z_j(t) &= z_k(t) \text{ for all } j, k = 1, \dots, N, \\ r_{j,\infty} &= \sqrt{\alpha/\beta} \quad \dot{\phi}_j = \omega - \gamma \sqrt{\alpha/\beta} \end{aligned}$$

represents a stable synchronized asymptotic state for the system, [20, 36], represented in the final column of Table 1. However, depending on the connectivity matrix, other stable asymptotic states representative of incoherence (uniformly distributed around a smaller limit cycle) and other symmetric configurations can coexist with the fully synchronized state, [2], represented in the second to last column of Table 1.

On the other hand, in the subcritical case, the amplitudes converge to zero exponentially fast, regardless of the coupling, while the phase behavior can be characterized by (6) where each $\frac{\dot{\phi}_j}{r_j}$ has converged to a finite value and thus can be absorbed into the network structure and each $r_j \rightarrow 0$ exponentially fast will no longer play a role in the phase dynamics [20]. Thus one can see that the fully synchronized state $\dot{\phi}_j = \omega$ can be recovered in this regime, as well as the incoherent and other symmetric states found in [44].

Now, focusing on the critical case. For the fully synchronized state, it is easy to see that if all $\phi_j = \phi_k$, and all $r_j = r_k$, then the respective dynamics are reduced to

$$\dot{r}_j = (\alpha - \beta r_j^2) r_j, \quad \text{and} \quad \dot{\phi}_j = \omega - \gamma r_j^2,$$

for each $j = 1, \dots, N$ and hence the critical value $\alpha = 0$ yields an algebraic speed of relaxation as in (4).

3.2 Kuramoto oscillators

Careful observation of (6) would lead the reader to notice the similarity to the famous Kuramoto model of phase synchronization. In fact, Kuramoto derived his model from the system (5) via a phase reduction that removed the amplitude dependence of the oscillators [17]. The reduction, which is valid in the weak coupling regime, involves taking the limits $\alpha, \beta \rightarrow \infty$ while keeping the ratio $\frac{\alpha}{\beta}$ fixed, forces each oscillator to take the amplitude $r_j = \sqrt{\alpha/\beta}$ so that the only relevant aspect of the model is the phase behavior. This is known as a phase reduction and gives rise to the model for identical Kuramoto oscillators (KO)

$$\dot{\phi}_j = \omega + \sum_{l=1}^N A_{lj} \sin(\phi_l - \phi_j).$$

We begin here with the coupled form as for an individual KO it is clear that $\dot{\phi} = \omega$ is the phase behavior, and there is no amplitude behavior, thus it always evolves on an orbit. From a mathematical perspective, this phase reduction greatly simplifies the model and allows for a more complete analysis of the dynamics. Under a symmetric graph structure, the model obeys a gradient flow structure given by

$$\begin{aligned} \dot{\Phi} &= -\nabla E(\Phi), \\ E(\Phi) &= \sum_{j \neq k=1}^N \left(\frac{1}{2} A_{jk} \cos(\phi_j - \phi_k) \right) - \sum_{j=1}^N \omega \phi_j \end{aligned}$$

The fully synchronized state represents the global minimum of the energy of the gradient flow and thus is always a steady state of the system and is exponentially stable. However, depending on the graph structure, other frequency synchronized states such that $\dot{\phi}_j = \dot{\phi}_k$ for all $j, k = 1, \dots, N$ while $\phi_j \neq \phi_k$ in general can also exist and be exponentially stable, e.g. for a ring structure [2]. In [48]

it was first shown that if a graph is dense enough, then the only stable outcome is the fully synchronized state, which is thus a global attractor, while in [44] it was shown that for a particular balanced graph structure, there are infinitely many distinct exponentially stable equilibria. Even further, under other specific graph structures there exist so-called twisted states and other balanced states which are not Lyapunov stable due to the existence of multiple zero eigenvalues, but nonetheless can be shown to be nonlinearly stable or neutrally stable [45, 51].

3.3 The Harmonic Oscillator

Our third type of oscillator is the Harmonic Oscillator (HO) which is given by a linear second order real ordinary differential equation

$$\ddot{x}(t) + 2\zeta\omega_0\dot{x}(t) + \omega_0^2x(t) = 0 \quad (7)$$

The value $\zeta \geq 0$ determines the damping of the system and yields 4 regimes.

- (1) (Undamped $\zeta = 0$): The solution is given by $x(t) = A \sin(\omega_0 t + \phi)$, where the amplitude $A > 0$ and phase ϕ are prescribed by the initial data.
- (2) (Underdamped $\zeta \in (0, 1)$): The solution is given by $x(t) = Ae^{-\zeta\omega_0 t} \sin(\omega_d t + \phi)$ where $\omega_d = \omega_0 \sqrt{1 - \zeta^2}$ slows the oscillation speed compared to the undamped case. The solution eventually stops as $x(t) \rightarrow 0$ exponentially fast at rate $\zeta\omega_0$.
- (3) (Critically damped $\zeta = 1$): The solution is given by $x(t) = (c_1 + c_2 t)e^{-\omega_0 t}$ as ω_0 is a repeated root of the characteristic equation and solutions converge to zero exponentially fast without oscillations.
- (4) (Overdamped $\zeta > 1$): The solution is given by $x(t) = c_1 e^{r_1 t} + c_2 e^{r_2 t}$ where r_1 and r_2 are negative real roots to the characteristic equation which drive the solution directly to zero with no oscillations.

As oscillations are the main object of concern, we do not further consider the critically damped or overdamped cases since they undergo exponential decay to zero without oscillation. The undamped case retains the first two regimes where $\zeta \in [0, 1)$ will be the focus. Case (1) can be identified with the second column of Table 1 as the lack of damping retains a constant amplitude. Case (2) on the other hand gives exponential decay to zero and thus the Amplitude Death and Exponential convergence columns of Table 1 are satisfied.

3.3.1 Coupled Harmonic Oscillators. Coupling identical oscillators of the form (7) gives the following equation

$$\ddot{x}_j + c\dot{x}_j + \sum_{l=1}^N A_{lj}(x_j - x_l) = 0.$$

For symmetric coupling A , it is classical, [35], to show that $x(t) = (x_1(t), \dots, x_n(t))$ is given by

$$\begin{aligned} x(t) &= \sum_{i=1}^N q_i(t) v_i, \quad q_i(t) = v_i^\top x(t), \\ q_i(t) &= e^{-\zeta_i \omega_i t} \left(q_i(0) \cos(\omega_{d,i} t) + \frac{\dot{q}_i(0) + \zeta_i \omega_i q_i(0)}{\omega_{d,i}} \sin(\omega_{d,i} t) \right), \end{aligned}$$

where (ω_i^2, v_i) are the respective eigenvalue-eigenvector pair of A , and $\zeta_i = \frac{c}{2\omega_i}$ and again $\omega_{d,i} = \omega_i \sqrt{1 - \zeta_i^2}$. In this way it is easy

to see that in the undamped case $c = 0$, the solution is given by $x(t) = \sum_i (A_i \cos(\omega_i t) + B_i \sin(\omega_i t))v_i$ with A_i and B_i completely determined by the initial data. On the other hand, in the damped case we see exponential decay to zero. Therefore both incoherence and synchronization are possible outcomes.

3.4 Phenomenological comparison of learning dynamics for the three oscillators

3.4.1 Individual oscillators: Active regimes: From a qualitative perspective the undamped regime ($\zeta = 0$) of the HO is most similar to the supercritical regime ($\alpha > 0$) of an SL oscillator as well as the behavior of a KO. Indeed, An individual KO oscillator and undamped HO have a constant phase velocity and no amplitude dynamics. If the initial data for an SL oscillator is not found on the stable limit-cycle, then the nonlinear damping term $-\beta|z(t)|^2 z(t)$ creates a correction mechanism for the amplitude behavior and a phase-drift that relaxes to the resulting limit-cycle determined by the system parameters $(\alpha, \beta, \omega, \gamma)$. In comparison, the KO has no amplitude behavior due to it being a purely phase-oscillator, while the amplitude of an HO is given by its initial data. This means that the SL oscillator is robust with respect to perturbations, while the HO yields periodic behavior once again, but at a new amplitude. The phase behavior of both the HO and SL oscillator are determined by the parameters however if the SL phase drift parameter γ is nonzero, then the final phase speed is affected by the amplitude behavior in contrast to the HO which has constant amplitude.

Decay regimes: A single underdamped HO with $\zeta \in (0, 1)$ undergoes oscillations, but exponentially decays to zero. The same is true for a subcritical ($\alpha < 0$) SL oscillator. However, the SL oscillator at critical value $\alpha = 0$ decays to zero at an algebraic rate. The KO is restricted to the torus and does not have amplitude dynamics.

3.4.2 The coupled systems: Active regimes: Starting with coupled undamped HOs, we see that the phase dynamics of each oscillator are entirely determined by the eigenvalues of the connectivity matrix A and that the amplitudes are still determined by the initial data. Further, as seen in [40], the energy of the system is preserved. Therefore synchronization only occurs if the connectivity matrix has all identical eigenvalues, and each node has identical initial data, otherwise the behavior will in general be quasiperiodic, and nodes should remain distinguishable.

Coupled Kuramoto oscillators have been seen to have a wealth of stable equilibria which are not synchronized states when the connectivity matrix is not too dense. However, the synchronized state always exists and is always exponentially stable with a large basin of attraction. Therefore as long as initial data is not found in the basin of attraction of the synchronized state the nodes should remain distinguishable, but there is a possibility for oversmoothing if perturbations lead to falling into the basin of attraction of the synchronized state.

As the Kuramoto model is a reduction of the Stuart-Landau system, we see that each of the stable equilibria of the KO model are also stable equilibria of the SL system, however, due to the evolution of SL in the complex plane rather than on a fixed torus there is not only the possibility of other fixed points, but a greater ability to explore the latent space before settling to the equilibria.

Decay regimes: As the Kuramoto model has no amplitude death, we only compare the underdamped HO ($c \in (0, 1)$) and (sub)critical SL oscillators ($\alpha \leq 0$) here. In these regimes we see both the SL oscillators and HO converge to zero. For all $c \in (0, 1)$ we see that oscillations continue, but each oscillator decays to zero at rate $\zeta_i = \frac{c}{2\omega_i}$ and hence one expects oversmoothing to occur. The same is true for subcritical ($\alpha < 0$) SL oscillators which decay to zero at an exponential rate. However, only the SL oscillatory system allows for even the synchronized state to avoid oversmoothing. In the critical case $\alpha = 0$, the amplitude dynamics converge to zero at an algebraic rate rather than exponential.

4 Start Landau Graph Neural Network

The Stuart-Landau Graph Neural Network (SLGNN) adapts the continuous-time dynamics of coupled Stuart-Landau oscillators for graph-based learning tasks. To achieve this, we formulate the node feature evolution as a graph ODE:

$$\dot{z}(t) = (\alpha + i\omega - (\beta + i\gamma)|z(t)|^2)z(t) + F_\theta(z(t), t),$$

where $z(0)$ is an initial feature matrix produced by a complex-valued MLP encoder. The first term governs the local Stuart-Landau dynamics for each node, while $F_\theta(z(t), t)$ represents a learnable, time-dependent nonlinear coupling between neighbouring nodes. However, a significant challenge in implementing this model is the numerical instability introduced by the cubic term $|z(t)|^2 z(t)$. Standard explicit numerical solvers, such as the Euler method, requires very small time steps to maintain stability, leading to model underperformance. While adaptive time-step solvers, such as the Dormand-Prince method, can handle such stiffness, they are often very slow and susceptible to step size underflow during training.

To overcome these issues, we propose a bespoke implicit-explicit (IMEX) numerical scheme. The core idea of an IMEX scheme is to treat the stiff parts of the ODE implicitly to ensure stability, while treating the non-stiff parts explicitly to maintain computational efficiency. A naive implicit approach might use Newton's method to solve the entire local oscillatory term

$$Z^{(l+1)} = Z^{(l)} + dt(\alpha + i\omega - (\beta + i\gamma)|Z^{(l+1)}|^2)Z^{(l+1)}, \quad (8)$$

while treating the coupling term explicitly. $Z^{(l)}$ denotes the complex-valued feature matrix at the l -th layer. Although feasible, this approach necessitates the computation of a 2D Jacobian and takes more iterations to converge. Therefore, we further refine the IMEX scheme by decomposing Equation (8) into their magnitude and phase terms, and only implicitly solve the stiff cubic magnitude term. This further reduces the dimensionality of the implicit part to one, as the phase update can be kept explicit. We present a Pseudocode of SLGNN in Algorithm 1.

The implicit update of $R^{(l+1)}$ can be efficiently computed using either 1D Newton's method or explicit inverse with Cardano's formula, using the real root of the corresponding cubic equation. We discuss the details in Appendix D. When Newton's method is used, it typically converges within 2 - 5 steps, depending on the set error tolerance. Both the encoder ENC and the decoder DEC are single-layered MLPs. While ENC is complex-valued, from our experience the choice between a real- or complex-valued DEC does not noticeably affect model performance. Other readout types including modulus, phases, and phase differences do not improve and

Algorithm 1 Pseudocode of SLGNN.

```

#  $X^{(0)}$ : initial real-valued node features
# ENC: complex-valued MLP encoder; DEC: real-valued MLP decoder
#  $F_\theta^{(l)}$ : coupling function;  $L$ : number of layers
#  $dt$ : time step;  $\alpha, \beta, \omega, \gamma$ : oscillator parameters

# Encode initial features into the complex domain
 $Z^{(0)} \leftarrow \text{ENC}(X^{(0)})$ 

# Iterate through  $L$  layers using the IMEX time-stepping scheme
for  $l = 0$  to  $L - 1$  do
    # Explicit step for the non-stiff coupling term
     $\dot{Z}^{(l)} \leftarrow Z^{(l)} + dt F_\theta^{(l)}(Z^{(l)})$ 

    # Decompose into magnitude and phase (Cartesian to Polar)
     $\tilde{R}^{(l)} \leftarrow \sqrt{\Re(\dot{Z}^{(l)})^2 + \Im(\dot{Z}^{(l)})^2}$ 
     $\tilde{\Phi}^{(l)} \leftarrow \text{atan2}(\Im(\dot{Z}^{(l)}), \Re(\dot{Z}^{(l)}))$ 

    # Implicitly solve for  $R^{(l+1)}$  with Newton's method or Cardano's formula
    Solve  $R^{(l+1)} = \tilde{R}^{(l)} + dt(\alpha - \beta(R^{(l+1)})^3)$ 

    # Explicitly update the phase using  $R^{(l+1)}$ 
     $\Phi^{(l+1)} \leftarrow \tilde{\Phi}^{(l)} + dt(\omega + \gamma(R^{(l+1)})^2)$ 

    # Recompose into a complex value (Polar to Cartesian)
     $Z^{(l+1)} \leftarrow R^{(l+1)} \cos(\Phi^{(l+1)}) + i \cdot R^{(l+1)} \sin(\Phi^{(l+1)})$ 
end for

# Decode the real component of the final layer's features
 $Y_{out} \leftarrow \text{DEC}(\Re(Z^{(L)}))$ 

Return  $Y_{out}$ 

```

may sometimes degrade model performance. Although this scheme requires repeated polar-cartesian conversion at each layer, empirically this introduces negligible computational overhead and does not impact model performance. This compares favorably with the alternative aforementioned 2D Newton's method, which does not necessitate such conversions but is more computationally intensive.

5 Experimental Evaluation

In this section, we conduct an extensive experimental evaluation of SLGNN against existing OGNs and the non-oscillatory baseline. For a fair comparison, we implement all models focusing on the core oscillatory mechanisms, without using additional model-specific artifacts. We extensively tune the hyperparameters of all models presented in this section, with the search spaces defined in Appendix A. We release the best hyperparameters along with the code (available upon publication).

5.1 Model Setups

Baselines. The baseline models in our study serve as non-oscillatory benchmarks for comparison. These models, which do not incorporate an explicit local oscillator term, also form the basis for the coupling functions in OGNs. The update of these models can be expressed as a forward Euler discretization of an ODE:

$$X^{(l+1)} = X^{(l)} + dt F_\theta^{(l)}(X^{(l)})$$

where $X^{(l)}$ is the matrix of node features at time t , dt is the step size, $F_\theta^{(l)}(X^{(l)})$ represents the nonlinear coupling with the neighboring nodes based on the graph structure. To ensure a fair comparison, we explicitly include a skip connection ($X^{(l)} + \dots$) in our baseline implementations, inherited from the Euler discretization. This makes the baselines more architecturally consistent with OGNs'

ODE structure. We highlight that in our experience, adding the skip connection consistently improves model performance. We consider three coupling functions, following a similar setup of prior works:

- **GCN:** The Graph Convolutional Network [16] coupling is defined as $F_\theta^{(l)}(X^{(l)}) = \sigma(D^{-\frac{1}{2}}AD^{-\frac{1}{2}}X^{(l)}W^{(l)})$, where σ is LeakyReLU with tunable negative gradient, A is the adjacency map of the graph augmented by self loops and D is the diagonal matrix such that $D_{ii} = \sum_j A_{ij}$. $W^{(l)}$ is a learnable weight matrix.
- **GAT:** The Graph Attention Network [50] coupling is defined as $F_\theta^{(l)}(X^{(l)}) = \sigma(A^{(l)}(X^{(l)}X^{(l)}W^{(l)}))$, where the adjacency matrix is computed with an attention mechanism. The entry for edge (i, j) is given by:

$$A(X^{(l)})_{ij} := \frac{\exp\left(\sigma(a^{(l)\top}[W^{(l)}X_i^{(l)}||W^{(l)}X_j^{(l)}])\right)}{\sum_{k \in \mathcal{N}_i} \exp\left(\sigma(a^{(l)\top}[W^{(l)}X_i^{(l)}||W^{(l)}X_k^{(l)}])\right)},$$

where $a^{(l)}$ is a learned $2m$ -dimensional vector and $X_i \in \mathbb{R}^m$ is the i -th row of the matrix X , i.e. the features of the node i , and finally $\mathcal{N}_i$ denotes the neighborhood of the node i in the graph $\mathcal{G}$.

- **Tran:** we also consider a feature-propagation-like setup similar to GRAND [3], where the feature update for each node is driven by a weighted sum of differences from its neighbors. Here the coupling function is defined as

$$F_\theta(X^{(l)}) = \sigma\left(\kappa \sum_{j \in \mathcal{N}_i} a_\theta^{(l)}(X_i, X_j)(X_j - X_i)\right),$$

$$a_\theta^{(l)}(X_i, X_j) = \text{softmax}\left(\frac{(W_K^{(l)}X_i)^\top(W_Q^{(l)}X_j)}{d_k}\right),$$

where κ is the coupling strength coefficient, $W_Q^{(l)}$ and $W_K^{(l)}$ are learnable query and key matrices, d_k is the key dimension.

Second Order Harmonic Oscillator (GraphCON [40]). In this case, the evolution of the features is governed by a second order ODE and solved with symplectic Euler:

$$\begin{aligned} Y^{(l+1)} &= Y^{(l)} + dt \sigma(F_\theta^{(l)}(X^{(l)})) - \gamma X^{(l)} - \alpha Y^{(l)}, \\ X^{(l+1)} &= X^{(l)} + dt Y^{(l+1)}, \end{aligned}$$

where the coupling $F_\theta^{(l)}$ represents the driving force, α models the angular frequency and γ the damping force. The variants GraphCON-GCN, GraphCON-GAT, and GraphCON-Tran are instantiated using the respective coupling functions $F_\theta^{(l)}$ described previously.

Kuramoto. The Kuramoto model treats the features of each node as the phase constrained on the surface of a hypersphere. The following Kuramoto architecture is proposed in [26]

$$X_i^{(l+1)} = X_i^{(l)} + dt \left(\omega + \kappa \sum_{k \in \mathcal{N}_i} a_\theta^{(l)}(X_i, X_k) \sin(X_k^{(l)} - X_i^{(l)}) \right), \quad (9)$$

where ω is the natural frequency, $\sin(X_k^{(l)} - X_i^{(l)})$ the nonlinear coupling term, and $a_\theta^{(l)}$ represents a learnable attention mechanism. However, directly solving Equation (9) with explicit Euler is implementation-wise challenging, as it is difficult to constrain the phase features within the $[0, 2\pi]$ interval, leading to instability during backpropagation and significant underperformance. While prior work [26] used adaptive step-size solvers, these can be computationally expensive and prone to step-size underflow. To address

Model	Cora	Citeseer	Pubmed
Baseline-GCN	82.35 ± 1.61	73.16 ± 1.94	78.24 ± 1.54
GraphCON-GCN	81.50 ± 1.57	72.83 ± 1.80	77.95 ± 1.30
Kuramoto-GCN	79.46 ± 2.05	70.83 ± 2.13	76.38 ± 3.72
SL-GCN	82.92 ± 1.39	73.29 ± 2.04	79.21 ± 1.30
Baseline-GAT	80.41 ± 1.76	72.95 ± 1.65	77.31 ± 1.67
Graphcon-GAT	81.64 ± 1.69	72.60 ± 1.79	77.33 ± 1.62
Kuramoto-GAT	80.63 ± 1.53	71.41 ± 2.08	77.38 ± 2.20
SL-GAT	81.97 ± 1.67	72.93 ± 1.79	77.55 ± 1.64
Baseline-Tran	83.27 ± 1.29	73.56 ± 1.78	78.60 ± 1.67
Graphcon-Tran	82.11 ± 1.41	72.51 ± 1.94	76.64 ± 1.85
Kuramoto-Tran	82.29 ± 1.22	72.97 ± 2.16	78.46 ± 1.45
SL-Tran	82.84 ± 1.50	74.43 ± 1.71	79.02 ± 1.76

Table 2: Transductive node classification accuracy (%) on homophilic datasets. The means and standard deviations are computed over 100 random dataset splits and initialization. Dark gray and light gray indicate the best and second best values, respectively.

this, as well as improving the adaptability of the model to different coupling functions, we propose an alternative numerical scheme by representing the phases as complex numbers on the unit circle $Z = e^{iX}$, and rewrite the layer update as the following

$$\begin{aligned}\tilde{Z}^{(l)} &= Z^{(l)} + dt F_{\theta}^{(l)}(Z^{(l)}), \\ \Phi^{(l+1)} &= \text{atan2}\left(\Im(\tilde{Z}^{(l)}), \Re(\tilde{Z}^{(l)})\right) + dt \omega, \\ Z^{(l+1)} &= \cos(\Phi^{(l+1)}) + \sin(\Phi^{(l+1)})i,\end{aligned}$$

where atan2 is the two-argument arctangent function. This is analogous to the IMEX scheme used for SLGNN, but without implicitly solving the magnitude term. Compared to adaptive step-size solvers like Dormand-Prince, this approach yields a runtime reduction of up to 100×, while achieving comparable or better performance. We also highlight that the proposed approach directly incorporates a generic learnable coupling function $F_{\theta}^{(l)}$, as opposed to (9).

5.2 Node Classification

We evaluate all models on transductive node classification tasks on both homophilic and heterophilic datasets, including Cora [19], Citeseer [46], Pubmed [24], WebKB [30] (Texas, Wisconsin, Cornell), WikipediaNetwork [38] (Chameleon, Squirrel), heterophilous graphs [32] (Amazon-rating). For homophilic graphs, we follow the data split schemes laid out by Shchur et al. [47]. As shown in Table 2 and 3, SLGNN variants consistently achieve competitive or state-of-the-art performance across both dataset types. Statistical significance tests are provided in Appendix B.

5.3 Graph-level tasks

To further evaluate performance of SLGNNs on graph-level tasks, We benchmark graph classification performance on the MoleculeNet datasets [52] (MUTAG, ENZYMES, PROTEINS), and graph regression on datasets from the Open Graph Benchmark [14, 15] (ESOL, FreeSolv, Lipophilicity). For these experiments, we adopt a

Model	Texas	Wisconsin	Cornell
Baseline-GCN	79.43 ± 5.58	84.39 ± 4.36	73.70 ± 4.17
GraphCON-GCN	78.49 ± 5.85	83.16 ± 4.43	71.97 ± 4.58
Kuramoto-GCN	80.57 ± 5.55	85.82 ± 3.82	74.11 ± 4.38
SL-GCN	81.81 ± 5.31	86.08 ± 4.45	74.46 ± 4.36
Baseline-GAT	79.00 ± 6.04	84.12 ± 4.30	72.92 ± 4.10
Graphcon-GAT	79.62 ± 6.22	82.73 ± 4.32	72.81 ± 3.95
Kuramoto-GAT	81.73 ± 5.67	85.27 ± 4.26	74.19 ± 4.30
SL-GAT	81.32 ± 5.41	85.96 ± 4.26	74.27 ± 3.91
Baseline-Tran	75.76 ± 6.44	83.08 ± 4.74	70.16 ± 5.85
Graphcon-Tran	82.89 ± 5.14	82.63 ± 4.87	72.73 ± 4.64
Kuramoto-Tran	78.70 ± 5.79	85.04 ± 4.48	72.43 ± 4.89
SL-Tran	85.16 ± 6.06	85.24 ± 4.44	76.27 ± 5.04

Model	Chameleon	Squirrel	Amazon-rating
Baseline-GCN	45.02 ± 2.20	34.52 ± 1.45	46.65 ± 1.86
GraphCON-GCN	47.83 ± 2.06	35.05 ± 1.46	48.07 ± 0.94
KuramotoGNN	49.39 ± 2.26	34.68 ± 1.27	48.65 ± 0.80
SL-GCN	49.77 ± 2.60	35.67 ± 1.49	51.27 ± 0.60
Baseline-GAT	46.58 ± 2.99	35.48 ± 1.59	49.83 ± 0.65
Graphcon-GAT	47.88 ± 2.60	36.35 ± 1.64	49.22 ± 1.31
Kuramoto-GAT	49.25 ± 2.13	34.53 ± 1.40	51.06 ± 0.55
SL-GAT	48.11 ± 2.18	35.75 ± 2.03	53.50 ± 0.42
Baseline-Tran	68.43 ± 2.22	59.68 ± 1.89	45.87 ± 0.54
Graphcon-Tran	66.88 ± 2.16	59.68 ± 2.50	46.08 ± 0.50
KuramotoGNN	65.32 ± 2.29	56.44 ± 1.79	46.17 ± 0.84
SL-Tran	66.03 ± 2.13	61.82 ± 2.29	48.64 ± 0.63

Table 3: Transductive node classification accuracy (%) on heterophilic datasets. The means and standard deviations are computed over 10 pre-defined splits, with 10 random initializations per split.

standard 80:10:10 train-validation-test split with a batch size of 64. As presented in Table 4 and 5, the results demonstrate the strong performance of SLGNNs on these tasks, which consistently outperform other OGNNs by up to 80% (see ENZYMES-Tran cells).

6 Discussions

Performance of deep GNNs. To assess the ability of different OGNNs to construct deep networks and mitigate oversmoothing, we evaluate their performance on Cora with GCN coupling across OGNNs of different depths. We note that the best hyperparameter configurations used in Table 2-5 may not translate to decent performance for deeper architectures. To ensure a fair comparison, we therefore re-tune the models for seven fixed depths ranging from 2 to 128 layers.

Figure 1 shows that SL-GCN maintains high performance even at significant depths, with its accuracy peaking at 32 layers. In contrast, the performances of both GraphCON and Kuramoto-GCN peak at 8 layers, with the latter rapidly collapsing to $29.98 \pm 2.02\%$ accuracy at 128 layers, indicating its weakness in deep settings. These findings align with our hyperparameter search for Table 2-5, where the optimal configurations for SLGNN frequently involved

Model	MUTAG	ENZYMES	PROTEINS
Baseline-GCN	70.47 ± 9.79	28.03 ± 7.27	70.50 ± 3.36
GraphCON-GCN	69.21 ± 10.95	29.87 ± 8.38	70.62 ± 4.26
Kuramoto-GCN	59.05 ± 10.59	29.27 ± 5.87	68.39 ± 4.50
SL-GCN	77.47 ± 9.00	44.33 ± 9.83	71.20 ± 4.56
Baseline-GAT	71.26 ± 11.87	44.57 ± 9.50	71.30 ± 4.42
Graphcon-GAT	71.89 ± 10.24	45.10 ± 11.23	71.98 ± 3.47
Kuramoto-GAT	71.58 ± 9.82	31.03 ± 8.79	71.30 ± 3.83
SL-GAT	77.79 ± 10.80	48.43 ± 9.14	70.89 ± 3.63
Baseline-Tran	61.26 ± 13.99	20.60 ± 4.87	69.77 ± 5.75
Graphcon-Tran	64.00 ± 11.88	17.70 ± 5.04	67.98 ± 4.75
Kuramoto-Tran	68.53 ± 12.30	21.90 ± 5.54	71.12 ± 4.09
SL-Tran	78.84 ± 13.54	39.40 ± 6.87	71.70 ± 3.30

Table 4: Graph classification accuracy (%) on MoleculeNet [52] datasets. The means and standard deviations are averaged over 50 random splits and weight initializations.

Model	ESOL	FreeSolv	Lipophilicity
Baseline-GCN	0.827 ± 0.199	4.780 ± 1.934	0.848 ± 0.462
GraphCON-GCN	0.956 ± 0.718	4.185 ± 1.468	0.498 ± 0.062
Kuramoto-GCN	0.757 ± 0.152	2.993 ± 1.311	0.562 ± 0.113
SL-GCN	0.673 ± 0.166	1.949 ± 0.819	0.495 ± 0.056
Baseline-GAT	0.565 ± 0.116	2.903 ± 1.926	0.684 ± 0.457
Graphcon-GAT	0.614 ± 0.125	5.938 ± 3.986	0.542 ± 0.176
Kuramoto-GAT	0.648 ± 0.128	2.130 ± 1.020	0.579 ± 0.082
SL-GAT	0.606 ± 0.139	2.014 ± 0.545	0.449 ± 0.069
Baseline-Tran	1.887 ± 0.629	7.406 ± 2.901	1.205 ± 0.173
Graphcon-Tran	1.799 ± 0.619	10.677 ± 3.176	1.426 ± 0.082
Kuramoto-Tran	1.943 ± 0.509	4.304 ± 1.484	0.879 ± 0.226
SL-Tran	1.187 ± 0.259	3.877 ± 1.783	0.695 ± 0.087

Table 5: Graph regression (L2 loss) on OGBG-MOL datasets [14, 15]. The means and standard deviations are averaged over 50 random splits and weight initializations.

deeper layers compared to GraphCON, Kuramoto, and the non-oscillatory baseline. This ability to effectively leverage network depth provides an intuition for the superior performance of SLGNNs over many evaluated tasks. Furthermore, it is also noteworthy that a simple Baseline-GCN with a skip connection component (added for consistency with the neural ODE framework but often omitted in the literature) can often achieve highly competitive performance against some of the more sophisticated OGNNs, even in very deep settings. Further discussion across model variants and datasets is presented in Appendix C.

Robustness to graph perturbations. Beyond depth scalability, we also evaluate model robustness against graph perturbations by randomly adding fake edges to the graph structure, following the protocol from Feng et al. [10]. As shown in Figure 2, SLGNN exhibits superior robustness compared to Kuramoto and achieves a level of robustness comparable to that of GraphCON.

Criticality of the Stuart Landau oscillator. A key property of the Stuart Landau Oscillator is its ability to operate near a critical state,

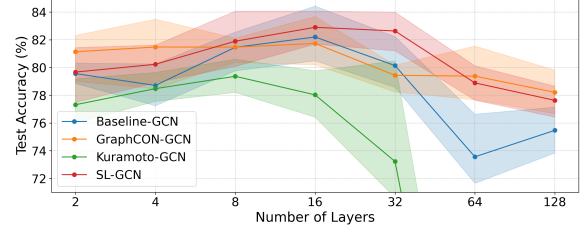


Figure 1: Node classification accuracy (%) on Cora for GNNs of varying depths. Shaded regions indicate the 10th to 90th percentile range. The mean values are computed over 10 random splits and weight initializations.

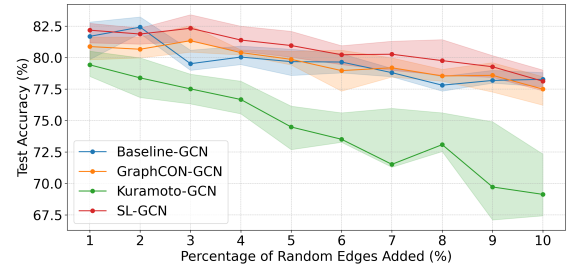


Figure 2: Robustness of GNNs on Cora dataset against random edge perturbations. Shaded regions indicate the 25th to 75th percentile range, computed over 10 random splits and weight initializations.

which has been extensively used by the neuroscience community for modeling the brain [34]. At this critical point, the system’s dynamics converge at a slower, algebraic rate rather than an exponential one. This state is reached when the following condition is met: for each node j ,

$$\alpha + \kappa \sum_{k=1}^N A_{jk} (\cos(\phi_j - \phi_k) \frac{r_k}{r_j} - 1) = 0.$$

Directly verifying this condition is challenging, as the coupling strength κ is implicitly set by the magnitude of the learned weights for SL-GCN and -GAT, and A_{jk} is dependent on the attention mechanism for SL-GAT and -Tran, both of which may not be fixed across layers. However, we find compelling empirical evidence that the model can leverage this property and reach a near-critical state. For instance, in our experiments with SL-GCN on Cora, Citeseer, and Pubmed, the hyperparameter tuner consistently selects a Hopf-parameter α close to 0 (e.g. 0.04), while we concurrently observe the average absolute value of $\cos(\phi_j - \phi_k) \frac{r_k}{r_j}$ remains around 1.0 (e.g. 1.06). In this special case, the oscillators appear to be almost synchronized, but with $\alpha \sim 0$, the Stuart-Landau oscillator is near its critical state. This suggests the hyperparameter tuner has leveraged the slower algebraic convergence to improve the performance of the model, a characteristic that no other OGNN frameworks have. This type of criticality-seeking behaviour appears to align with

the idea that the brain operates near criticality [23]. The precise implications in deep learning will be the topic of a future study.

Computational Cost. A practical consideration for SLGNN is its computational cost. The bespoke IMEX time-stepping scheme, necessary to handle the stiff cubic term in the Stuart-Landau dynamics, incurs an overhead compared to the fully explicit methods used for Kuramoto and GraphCON models. Consequently, SLGNN's training times are typically about double those of other OGNN counterparts. However, our IMEX implementation is highly optimized: by reducing the implicit part to a one-dimensional problem, it achieves a runtime improvement by over two orders of magnitude compared to standard adaptive solvers like the Dormand-Prince method. This approach not only makes the model computationally feasible but also circumvents issues like time-step underflow while maintaining excellent numerical stability.

7 Conclusion

In this work, we introduced the Stuart-Landau Graph Neural Network (SLGNN), a novel oscillatory architecture grounded in the rich dynamics of Stuart-Landau systems. By modeling the coupled evolution of both feature amplitude and phase, SLGNN generalizes prior phase-only models and gains significant expressive power. Aided by a bespoke implicit-explicit numerical scheme for stable and efficient training, our extensive experiments on node, graph, and regression tasks demonstrate that this Stuart-Landau-based approach translates into state-of-the-art performance, outperforming existing oscillatory GNNs.

8 Acknowledgement

KZ was supported by the EPSRC Centre for Doctoral Training in Mathematical Modelling, Analysis and Computation (MAC-MIGS) funded by the UK Engineering and Physical Sciences Research Council (grant EP/S023291/1), Heriot-Watt University and the University of Edinburgh. DNR was funded by National Science Centre, Poland, grant number 2023/50/A/ST1/00447 and partially supported by the Modeling Nature (MNat) Research Unit, project QUAL21-011. PD and FT are members of the Gruppo Nazionale Calcolo Scientifico - Istituto Nazionale di Alta Matematica (GNCS-INdAM). FT is partially funded by the PRIN-MUR project MOLE (code 2022ZK5ME7) and the PRIN-PNRR project FIN4GEO (code P2022BNB97). PD was supported by the INdAM-GNCS project "NLA4ML—Numerical Linear Algebra Techniques for Machine Learning". This work was supported by the Edinburgh International Data Facility (EIDF) and the Data-Driven Innovation Programme at the University of Edinburgh, as well as AI Research Resource Programme (AIRR) under UK Research and Innovation (UKRI).

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Knowledge Discovery and Data Mining* (2019-07-25). arXiv: arXiv:1907.10902 [cs] doi:10.1145/3292500.3330701
- [2] Alex Arenas, Albert Diaz-Guilera, Jurgen Kurths, Yamir Moreno, and Changsong Zhou. 2008. Synchronization in complex networks. *Physics Reports* 469 (2008), 93–153. Issue 3. doi:10.1016/j.physrep.2008.09.002. (https://www.sciencedirect.com/science/article/pii/S0370157308003384)
- [3] Benjamin Paul Chamberlain, James Rowbottom, Maria Gorinova, Stefan Webb, Emanuele Rossi, and Michael M. Bronstein. 2021. GRAND: Graph Neural Diffusion. In *ICML*. arXiv:2106.10934 [cs, stat]
- [4] Jeongwhan Choi, Seoyoung Hong, Noseong Park, and Sung-Bae Cho. 2023. GREAD: Graph Neural Reaction-Diffusion Networks. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 5722–5747. https://proceedings.mlr.press/v202/choi23a.html
- [5] Andreas Damianou, Francesco Fabbri, Paul Gigoli, Marco De Nadai, Alice Wang, Enrico Palumbo, and Mounia Lalmas. 2024. Towards Graph Foundation Models for Personalization. In *WWW*. arXiv:2403.07478 [cs] doi:10.48550/arXiv.2403.07478
- [6] Moshe Eliasof, Eldad Haber, and Eran Treister. 2021. PDE-GCN: Novel Architectures for Graph Neural Networks Motivated by Partial Differential Equations. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 3836–3849. https://proceedings.neurips.cc/paper_files/paper/2021/file/1f9fd8ff75205aa73ec83e543d8b571-Paper.pdf
- [7] Moshe Eliasof, Eldad Haber, and Eran Treister. 2024. Feature transportation improves graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 11874–11882.
- [8] Moshe Eliasof, Eldad Haber, and Eran Treister. 2024. Graph Neural Reaction Diffusion Models. *SIAM Journal on Scientific Computing* 46, 4 (2024), C399–C420. arXiv:https://doi.org/10.1137/23M1576700 doi:10.1137/23M1576700
- [9] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *WWW*. arXiv:1902.07243 [cs] doi:10.1145/3308558.3313488
- [10] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph Random Neural Networks for Semi-Supervised Learning on Graphs. (2020). arXiv:2005.11079 [cs] doi:10.48550/arXiv.2005.11079
- [11] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1263–1272. https://proceedings.mlr.press/v70/gilmer17a.html
- [12] Francesco Di Giovanni, T. Konstantin Rusch, Michael Bronstein, Andreea Deac, Marc Lackenby, Siddhartha Mishra, and Petar Veličković. 2024. How does over-squashing affect the power of GNNs? *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=KJRQvRWns
- [13] Kai Guo, Xiaofeng Cao, Zhining Liu, and Yi Chang. 2023. Taming Over-Smoothing Representation on Heterophilic Graphs. *Information Sciences* 647 (Nov. 2023), 119463. doi:10.1016/j.ins.2023.119463
- [14] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. abs/2005.00687 (2020). arXiv:2005.00687 [cs] doi:10.48550/arXiv.2005.00687
- [15] Sergei Ivanov, Sergei Sviridov, and Evgeny Burnaev. 2019. Understanding Isomorphism Bias in Graph Data Sets. abs/1910.12091 (2019). arXiv:1910.12091 [cs] doi:10.48550/arXiv.1910.12091
- [16] Thomas N. Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*. arXiv:1609.02907
- [17] Yoshiki Kuramoto. 1975. Self-entrainment of a population of coupled non-linear oscillators. *Lect. Notes Phys.* 39 (1975), 420–422. doi:10.1007/BFb0013365
- [18] Samuel Lanthaler, T. Konstantin Rusch, and Siddhartha Mishra. 2023. Neural Oscillators Are Universal. In *NeurIPS*. arXiv:2305.08753 [cs] doi:10.48550/arXiv.2305.08753
- [19] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the Construction of Internet Portals with Machine Learning. 3, 2 (2000), 127–163. doi:10.1023/A:1009953814988
- [20] Ana P. Millán, David Poyato, David N. Reynolds, and Francesco Tudisco. 2025. Synchronization of coupled Stuart-Landau oscillators: How heterogeneity can facilitate synchronization. arXiv:2510.05243 (2025).
- [21] Takeru Miyato, Sindy Löwe, Andreas Geiger, and Max Welling. 2024. Artificial Kuramoto Oscillatory Neurons. arXiv:2410.13821 (Oct. 2024). arXiv:2410.13821 doi:10.48550/arXiv.2410.13821
- [22] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilbol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2017. Ray: A Distributed Framework for Emerging AI Applications. In *USENIX Symposium on Operating Systems Design and Implementation* (2017-12-16), Vol. abs/1712.05889. arXiv. arXiv:1712.05889 [cs] doi:10.48550/arXiv.1712.05889
- [23] Miguel A. Muñoz. 2018. Colloquium: Criticality and dynamical scaling in living systems. *Rev. Mod. Phys.* 90 (Jul 2018), 031001. Issue 3. doi:10.1103/RevModPhys.90.031001
- [24] Galileo Namata, Ben London, L. Getoor, and Bert Huang. 2012. Query-Driven Active Surveying for Collective Classification. https://www.semanticscholar.org/paper/efac04450c531b3769451a886ed9a42f2c4754d9
- [25] Khang Nguyen, Hieu Nong, Vinh Nguyen, Nhat Ho, Stanley Osher, and Tan Nguyen. 2022. Revisiting Over-Smoothing and over-Squashing Using Ollivier's

- Ricci Curvature. In *ICML*. arXiv:2211.15779 [cs, stat] doi:10.48550/arXiv.2211.15779
- [26] Tuan Nguyen, Hirotada Honda, Takashi Sano, Vinh Nguyen, Shugo Nakamura, and Tan M. Nguyen. 2023. From Coupled Oscillators to Graph Neural Networks: Reducing Over-smoothing via a Kuramoto Model-based Approach. *International Conference on Artificial Intelligence and Statistics* (Nov. 2023), 2710–2718. arXiv:2311.03260 [cs] doi:10.48550/arXiv.2311.03260
- [27] Hoang Nt and Takanori Maehara. 2019. Revisiting Graph Neural Networks: All We Have Is Low-Pass Filters. arXiv:1905.9550 (May 2019). arXiv:1905.9550 [stat] doi:10.48550/arXiv.1905.09550
- [28] Kenta Oono and Taiji Suzuki. 2019. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *ICLR*. arXiv:1905.10947 [cs, stat]
- [29] Elena Panteley, Antonio Loria, and Ali El Ati. 2015. On the Stability and Robustness of Stuart-Landau Oscillators. *IFAC-PapersOnLine* 48, 11 (2015), 645–650. doi:10.1016/j.ifacol.2015.09.260
- [30] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *ICLR*. arXiv:2002.05287 [cs] doi:10.48550/arXiv.2002.05287
- [31] Shaowen Peng, Kazunari Sugiyama, and Tsunenori Mine. 2022. SVD-GCN: A Simplified Graph Convolution Paradigm for Recommendation. In *CIKM*. arXiv:2208.12689 [cs] doi:10.1145/3511808.3557462
- [32] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A Critical Look at the Evaluation of GNNs under Heterophily: Are We Really Making Progress?. In *ICLR*. arXiv:2302.11640 [cs] doi:10.48550/arXiv.2302.11640
- [33] Michael Poli, Stefano Massaroli, Junyoung Park, Atsushi Yamashita, Hajime Asama, and Jinkyoo Park. 2019. Graph neural ordinary differential equations. *arXiv preprint arXiv:1911.07532* (2019).
- [34] A Ponce-Alvarez and G Deco. 2024. The Hopf whole-brain model and its linear approximation. *Sci Rep* 14 (2024), 2615. doi:10.1038/s41598-024-53105-0
- [35] S.S. Rao. 2004. *Mechanical Vibrations*. Pearson Prentice Hall.
- [36] David N Reynolds and Francesco Tudisco. 2025. Unique Nash equilibrium of a nonlinear model of opinion dynamics on networks with friction-inspired stubbornness. *Nonlinearity* 38, 5 (may 2025), 055027. doi:10.1088/1361-6544/add3ae
- [37] Andreas Roth and Thomas Liebig. 2023. Rank Collapse Causes Over-Smoothing and over-Correlation in Graph Neural Networks. In *LoG*.
- [38] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2019. Multi-Scale Attributed Node Embedding. 9, 2 (2019), cnab014. arXiv:1909.13021 [cs] doi:10.1093/comnet/cnab014
- [39] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. 2023. A Survey on Oversmoothing in Graph Neural Networks. arXiv:2303.10993 (March 2023). arXiv:2303.10993 [cs] doi:10.48550/arXiv.2303.10993
- [40] T. Konstantin Rusch, Benjamin P. Chamberlain, James Rowbottom, Siddhartha Mishra, and Michael M. Bronstein. 2022. Graph-Coupled Oscillator Networks. In *ICML*. arXiv:2202.02296 [cs, math, stat]
- [41] T. Konstantin Rusch and Siddhartha Mishra. 2020. Coupled Oscillatory Recurrent Neural Network (coRNN): An Accurate and (Gradient) Stable Architecture for Learning Long Time Dependencies. In *ICLR*. arXiv:2010.00951 [cs] doi:10.48550/arXiv.2010.00951
- [42] T. Konstantin Rusch and Siddhartha Mishra. 2021. UniCORNN: A Recurrent Model for Learning Very Long Time Dependencies. In *ICML*. arXiv:2103.05487 [cs] doi:10.48550/arXiv.2103.05487
- [43] T. Konstantin Rusch and Daniela Rus. 2024. Oscillatory State-Space Models. abs/2410.03943 (2024). arXiv:2410.03943 [cs] doi:10.48550/arXiv.2410.03943
- [44] Davide Sclosa. 2023. Kuramoto Networks with Infinitely Many Stable Equilibria. *SIAM Journal on Applied Dynamical Systems* 22, 4 (2023), 3267–3283. arXiv:https://doi.org/10.1137/23M155400X doi:10.1137/23M155400X
- [45] Davide Sclosa. 2024. Completely degenerate equilibria of the Kuramoto model on networks. *Nonlinearity* 37, 9 (aug 2024), 095021. doi:10.1088/1361-6544/ad694a
- [46] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. 29, 3 (2008), 93–93. doi:10.1609/aimag.v29i3.2157
- [47] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. arXiv:1811.5868 (Nov. 2018). arXiv:1811.5868 [cs] doi:10.48550/arXiv.1811.05868
- [48] Richard Taylor. 2012. There is no non-zero stable fixed point for dense networks in the homogeneous Kuramoto model. *Journal of Physics A: Mathematical and Theoretical* 45, 5 (jan 2012), 055102. doi:10.1088/1751-8113/45/5/055102
- [49] Matthew Thorpe, Hedi Xia, Tan Nguyen, Thomas Strohmer, Andrea L Bertozzi, and Stanley J Osher. 2022. Grand++: Graph Neural Diffusion with a Source Term. (2022).
- [50] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. Graph Attention Networks. In *ICLR*. arXiv:1710.10903 [cs, stat] doi:10.17863/CAM.48429
- [51] Daniel A. Wiley, Steven H. Strogatz, and Michelle Girvan. 2006. The size of the sync basin. *Chaos: An Interdisciplinary Journal of Nonlinear Science* 16, 1 (03 2006), 015103. doi:10.1063/1.2165594
- [52] Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. 2017. MoleculeNet: A Benchmark for Molecular Machine Learning. 9 (2017), 513–530. arXiv:1703.00564 [cs] doi:10.1039/c7sc02664a
- [53] Louis-Pascal A. C. Xhonneux, Meng Qu, and Jian Tang. 2019. Continuous Graph Neural Networks. In *ICML*. arXiv:1912.00967 [cs] doi:10.48550/arXiv.1912.00967
- [54] Kaicheng Zhang, Piero Deidda, Desmond Higham, and Francesco Tudisco. 2025. Rethinking Oversmoothing in Graph Neural Networks: A Rank-Based Perspective. *arXiv preprint arXiv:2502.04591* (2025).

Model	Cora	Citeseer	Pubmed	Texas	Wisconsin	Cornell
SL-GCN	2.67	0.46	4.81	1.61	0.44	0.57
SL-GAT	1.39	1.30	0.62	2.06	1.15	0.14
SL-Tran	2.84	3.52	1.73	2.86	0.32	5.17

Model	Chameleon	Squirrel	Amazon-rating	MUTAG	ENZYMES	PROTEINS
SL-GCN	1.10	2.97	26.20	3.72	7.92	0.66
SL-GAT	0.68	1.05	35.26	2.80	1.63	-0.51
SL-Tran	-2.80	7.21	23.52	3.99	14.02	0.78

Model	ESOL	FreeSolv	Lipophilicity
SLGCN	4.20	9.41	3.76
SLGAT	0.30	3.14	8.58
SLTran	7.28	7.33	18.62

Table 6: T-test scores of the results in Table 2-5. For regression tasks (ESOL, FreeSolv, Lipophilicity), we present the negative t-score.

A Hyperparameter Tuning Setup

We conduct extensive hyperparameter search across all model-dataset pairs presented in Table 2-5 with Ray platform [22] and Optuna [1] search algorithm over 1000 trials for node-level tasks and 500 for graph-level ones. The search spaces for all GCN, GAT, and Tran variants include: latent dimension size [16, 128], number of layers [1, 80], learning rate [10^{-5} , 10^{-2}], weight decay [10^{-6} , 1.0], LeakyReLU gradient [0.0, 1.0], dropout and input dropout rates [0.0, 0.5]. The time step size is fixed at $dt = 1.0$. For GAT and Tran, we fix the number of attention heads to 2 or 4. For Tran, we additionally tune the attention dimension between [4, 80] and the coupling strength κ in [0.1, 1.5]. For the OGNNs, we tuned their specific parameters:

- For SLGNN, we search α in $[-0.5, 0.5]$, β in $[0.0, 2.0]$, ω in $[0.5, 2.0]$ and γ in $[-1.0, 2.0]$.
- For Kuramoto, we tune the natural frequency ω in $[0.0, 2.0]$, which is consistent with that of SLGNN.
- For GraphCON, we tune both α and γ in $[0.0, 1.5]$, consistent with the search space used in [40].

Hyperparameters are optimized by maximizing classification accuracy or minimizing regression loss on the validation set. We report the final evaluation results on the test set with best-performing configuration on the validation set.

B Statistical Significance Tests

To validate the results presented in Table 2-5, we conduct one-tailed t-tests to assess the statistical significance of the performance improvements achieved by our SLGNN models. We use a 95% confidence level for all tests. For each dataset and model variant (GCN, GAT, Tran), we compare the best-performing SLGNN against the second best-performing model. In the few cases where SLGNN is the second-best performer, we compare it against the third-best one. The t-score is calculated as follows:

$$\text{t-score} = \frac{\mu_{\text{SL}} - \mu_{\text{other}}}{\sqrt{\frac{\sigma_{\text{SL}}^2}{n} + \frac{\sigma_{\text{other}}^2}{n}}},$$

where μ and σ^2 represent the mean and variance of the model performance over n number of repeated trials. A t-score greater than 1.66 indicates that SLGNN's performance is significantly superior with 95% confidence. As presented in Table 6, the results show that SLGNN achieve statistically significant gains in many model variant-dataset pairs, providing strong evidence for the effectiveness of the proposed architecture.

C Additional Results on Model Depths

This appendix extends the discussion from Section 6 on the relationship between network depth and model performance. In Figure 3, we present the distribution of the optimal model depths selected by hyperparameter tuning algorithm.

We note that SLGNN's optimal configurations are less concentrated in the shallow 1- to 10-layer range compared to GraphCON, Kuramoto, and the Baseline. This reinforces our claim that SLGNNs typically leverage deeper network structures to achieve better performance. On the other hand, there are a significant number of Kuramoto's configurations in the very deep 70- to 80-layer range. At first glance, this may seem to contradict the conclusions drawn from Figure 1. However, we observe that models with optimal depths in this range are predominantly those using Tran coupling, a configuration that makes models more robust at greater depths comparing to the GCN or GAT coupling. It is also known that certain datasets, such as the heterophilic ones, are less susceptible to performance degradation in deep architectures [13]. Most importantly, increased depth does not always translate to better performance, particularly for the Baseline and other OGNN models.

To quantify this relationship, we calculated the correlation coefficient between network depth used in the best hyperparameter configuration and model accuracy across all node and graph classification tasks in Table 7. The results show that SLGNN has a substantially stronger positive correlation between depth and performance than other model classes. This indicates that SLGNN is more effective at leveraging additional layers for performance gains. Therefore, while it may not always require maximum depth, its ability to effectively use deeper architectures is superior, again reinforcing the conclusions from our main analysis.

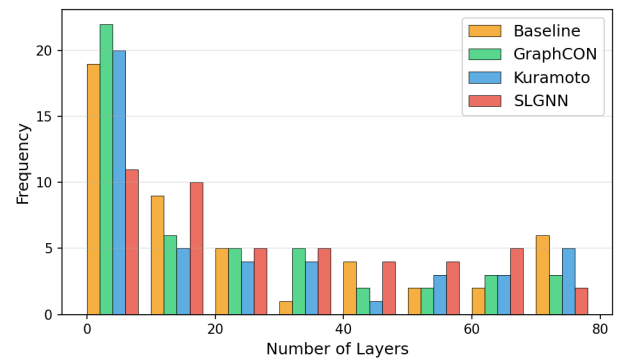


Figure 3: Distribution of optimal layer depths selected by the hyperparameter tuner for different model types.

	Baseline	GraphCON	Kuramoto	SLGNN
Correlation Coefficient	0.223	0.148	0.159	0.313

Table 7: Correlation coefficients between the optimal layer depths and the model accuracies on the node and graph classification tasks from Table 2-4. For each dataset, the accuracy is normalized by the worst performing accuracy from any model to set a performance baseline of 1.

D Implicit Solving Magnitude Equation

This appendix discusses using Newton’s method or explicit inverse to implicitly solve the magnitude component in Algorithm 1,

$$R^{(l+1)} = \tilde{R}^{(l)} + dt(\alpha - \beta(R^{(l+1)})^3).$$

For Newton, one can define

$$F(R^{(l+1)}, \tilde{R}^{(l)}) = R^{(l+1)} - \tilde{R}^{(l)} - dt\dot{r}(R^{(l+1)}) = 0,$$

$$\frac{\partial F(R^{(l+1)}, \tilde{R}^{(l)})}{\partial R^{(l+1)}} = 1 - dt(\alpha - 3\beta(R^{(l+1)})^2).$$

The Newton update can then be expressed as

$$R^{(l+1)} \leftarrow R^{(l+1)} - \left(\frac{\partial F(R^{(l+1)}, \tilde{R}^{(l)})}{\partial R^{(l+1)}} \right)^{-1} F(R^{(l+1)}, \tilde{R}^{(l)}).$$

The iteration stops when the maximum component of subsequent update is less than a pre-set tolerance. We typically set this tolerance to 10^{-5} , although larger values can also be used to marginally reduce runtime without noticeable drop in model performance.

For explicit inverse, we re-write the above equation in its depressed form

$$\left(R^{(l+1)}\right)^3 + \left(\frac{1-dt\alpha}{dt\beta}\right)R^{(l+1)} + \left(\frac{-\tilde{R}^{(l)}}{dt\beta}\right) = 0,$$

where we let $p = (1 - dt\alpha)/(dt\beta)$, $q = -\tilde{R}^{(l)}/(dt\beta)$, and define

$$u_{\pm} = -\frac{q}{2} \pm \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}, \quad \epsilon_{\pm} = \frac{-1 \pm i\sqrt{3}}{2},$$

where $\epsilon_{\pm}$ are the roots of unity. The three roots of the cubic equation are then given by Cardano’s formula: $\sqrt[3]{u_+} + \sqrt[3]{u_-}$ (always real), $\epsilon_+ \sqrt[3]{u_+} + \epsilon_- \sqrt[3]{u_-}$ and $\epsilon_- \sqrt[3]{u_+} + \epsilon_+ \sqrt[3]{u_-}$. The latter two roots are real if the condition $(q/2)^2 + (p/3)^3 > 0$ is satisfied. In most cases, this equation typically only has one real root, unless the feature values are exploding due to a poorly chosen set of hyperparameters.

E Computational Complexity

The computational complexity of the proposed SLGNN is consisted of the coupling part and the IMEX part. The coupling complexity depends on the specific coupling function used. For instance, a GCN coupling scales as $O(|E|d + |V|d^2)$, where $|V|$ and $|E|$ denote the number of nodes and edges, respectively, and d represents the feature dimension.

Regarding the IMEX scheme, the implicit step using Newton’s method has a per-feature, per-iteration complexity of $O(1)$. Consequently, the complexity of the IMEX scheme over the entire graph is $O(k|V|d)$, where k represents the number of Newton iterations. Our empirical observations suggest that k depends primarily on the feature distribution rather than the graph size.

Empirically, SLGNN is observed to be up to $2\times$ slower per epoch than Kuramoto and up to $3\times$ slower than GraphCON for the best hyperparameter configurations as shown in Table 8. However, it remains substantially faster than the explicit high-order ODE solvers used in prior works, such as the Dormand Prince method. In terms of memory efficiency, the VRAM usage of SLGNN is comparable to Kuramoto and within $1.5\times$ of GraphCON as shown in Table 9. It is important to note that SLGNN often leverages deeper architectures to achieve optimal performance as shown in Figure 3, which naturally contributes to increased computation time. This additional computational cost is counterbalanced by the model’s superior predictive performance.

Table 8: Runtime per epoch (ms) comparison. Values represent mean $\pm$ standard deviation.

Model	Cora	Citeseer	Pubmed
Baseline-GCN	9.62 $\pm$ 0.74	11.10 $\pm$ 0.38	18.39 $\pm$ 0.55
GraphCON-GCN	11.94 $\pm$ 0.51	13.68 $\pm$ 1.13	20.42 $\pm$ 0.56
Kuramoto-GCN	16.22 $\pm$ 0.48	19.00 $\pm$ 0.31	24.73 $\pm$ 0.41
SL-GCN	32.82 $\pm$ 2.32	32.40 $\pm$ 1.10	41.22 $\pm$ 1.79

Model	Texas	Wisconsin	Cornell
Baseline-GCN	10.40 $\pm$ 0.72	11.59 $\pm$ 0.53	11.16 $\pm$ 0.49
GraphCON-GCN	12.92 $\pm$ 0.57	13.73 $\pm$ 0.24	13.43 $\pm$ 0.57
Kuramoto-GCN	18.67 $\pm$ 0.81	18.76 $\pm$ 0.40	18.95 $\pm$ 0.70
SL-GCN	37.59 $\pm$ 1.89	36.98 $\pm$ 1.20	39.85 $\pm$ 1.85

Table 9: Peak VRAM Usage (MB) comparison. Values represent mean $\pm$ standard deviation.

Model	Cora	Citeseer	Pubmed
Baseline-GCN	66.42 $\pm$ 0.00	98.50 $\pm$ 0.00	262.79 $\pm$ 0.00
GraphCON-GCN	68.26 $\pm$ 0.00	97.53 $\pm$ 0.00	278.06 $\pm$ 0.00
Kuramoto-GCN	129.34 $\pm$ 0.00	259.35 $\pm$ 0.00	390.16 $\pm$ 0.00
SL-GCN	123.06 $\pm$ 2.74	179.64 $\pm$ 5.61	574.39 $\pm$ 10.20

Model	Texas	Wisconsin	Cornell
Baseline-GCN	22.92 $\pm$ 0.00	26.75 $\pm$ 0.00	27.14 $\pm$ 0.00
GraphCON-GCN	22.42 $\pm$ 0.00	26.19 $\pm$ 0.00	27.31 $\pm$ 0.00
Kuramoto-GCN	27.41 $\pm$ 0.00	33.21 $\pm$ 0.00	32.30 $\pm$ 0.00
SL-GCN	31.32 $\pm$ 2.14	36.12 $\pm$ 2.55	34.75 $\pm$ 1.41